

OPEN TECH NOTES

loạt tài liệu kỹ thuật miễn phí cho cộng đồng

AI AGENTS

Khi mô hình không chỉ trả lời, mà tự hành động

Vòng lặp reasoning-acting, kiến trúc đa tác nhân, và rủi ro khi trao quyền cho AI

Phần 6 trong loạt bài Kỹ thuật AI Engineering

Biên soạn cho cộng đồng IT Việt Nam · Tháng 8, 2026

Mục lục

Mở đầu	3
1. AI Agent là gì	3
2. Thành phần cốt lõi của một agent	3
2.1. Vòng lặp reasoning – acting (ReAct)	3
2.2. Function calling / tool use	4
2.3. Bộ nhớ (memory)	4
3. Kiến trúc đa tác nhân (multi-agent)	5
4. Đánh giá và gỡ lỗi agent	5
5. Rủi ro & giới hạn thực tế	6
5.1. Lỗi tích lũy qua nhiều bước	6
5.2. Quyền hạn & an toàn	6
6. Checklist nhanh cho đội ngũ	6
Đọc thêm	7
Tài liệu tham khảo	7

Mở đầu

Năm phần trước của loạt bài đã đi qua các công cụ nền tảng để làm việc với foundation model: chọn mô hình, viết prompt tốt, cấp kiến thức bên ngoài qua RAG, tùy chỉnh hành vi qua fine-tuning, và đo lường chất lượng một cách nghiêm túc. Tất cả những công cụ đó có một điểm chung: mô hình nhận một đầu vào, sinh ra một đầu ra, rồi dừng lại con người là bên quyết định bước tiếp theo.

AI agent thay đổi mô hình đó: thay vì chỉ trả lời một lần, mô hình được trao khả năng tự quyết định chuỗi hành động cần làm để đạt một mục tiêu tìm kiếm thông tin, gọi API, chạy mã, đọc/ghi file quan sát kết quả, và tự điều chỉnh bước tiếp theo, lặp lại cho đến khi hoàn thành nhiệm vụ. Đây là phần khép lại loạt bài, vì một agent thực chất là sự kết hợp của tất cả những gì năm phần trước đã trình bày: một mô hình nền đủ mạnh, được dẫn dắt bởi prompt tốt, có thể truy xuất kiến thức qua RAG, đôi khi được fine-tune cho hành vi chuyên biệt, và luôn cần được đánh giá cẩn thận trước khi trao quyền hành động thực sự. Tài liệu này trình bày AI agent là gì, các thành phần cốt lõi tạo nên một agent, kiến trúc đa tác nhân, và những rủi ro cần cân nhắc kỹ khi để mô hình tự hành động. Nội dung là bản tổng hợp gốc dựa trên kiến thức chung của ngành và các nguồn công khai, có trích dẫn ở cuối bài.

1. AI Agent là gì

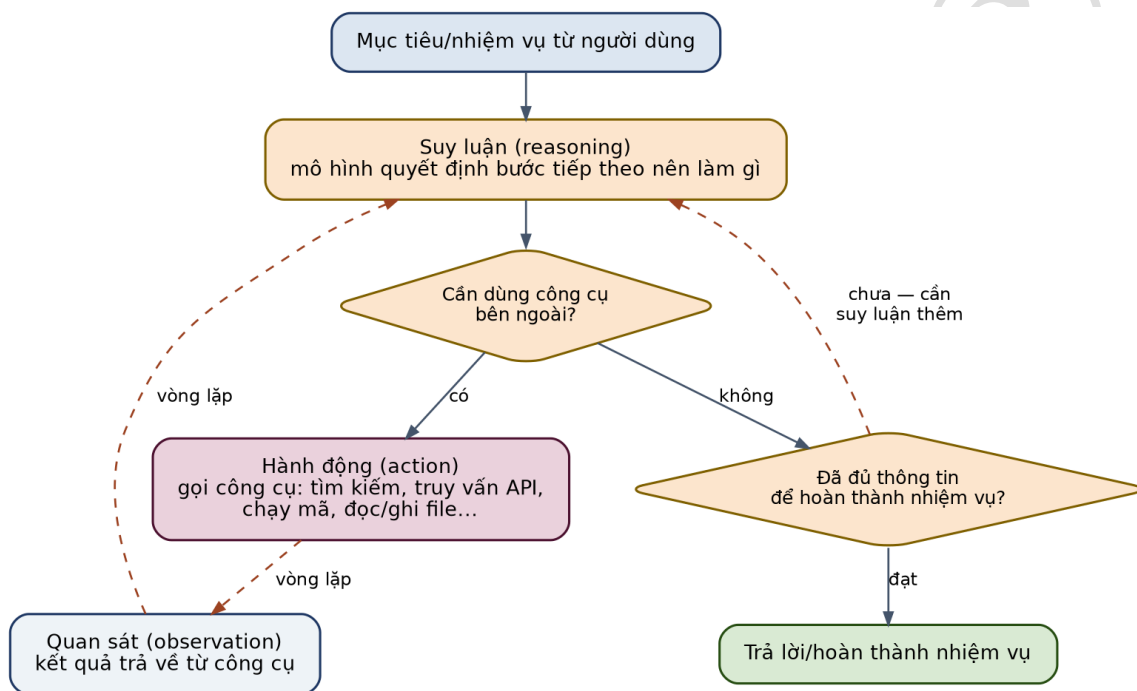
Một cách phân biệt hữu ích: một ứng dụng foundation model "thuần" (workflow) thực hiện một chuỗi bước đã được lập trình sẵn cố định kể cả khi có gọi công cụ ở giữa, trình tự các bước do con người thiết kế trước. Một AI agent, ngược lại, tự mô hình quyết định trình tự bước kể cả có bao nhiêu bước, dùng công cụ nào, và khi nào dừng dựa trên mục tiêu được giao và kết quả quan sát được ở mỗi bước, không phải theo một kịch bản cố định.

Ranh giới giữa hai khái niệm này không tuyệt đối nhiều hệ thống thực tế nằm ở giữa, kết hợp một khung quy trình cố định với một vài bước có tính "tự chủ" cục bộ. Nguyên tắc thực dụng thường được khuyến nghị (Anthropic, 2024): bắt đầu với giải pháp đơn giản nhất có thể prompt trực tiếp, hoặc một workflow cố định và chỉ tăng mức độ tự chủ của agent khi độ phức tạp của nhiệm vụ thực sự đòi hỏi, vì agent càng tự chủ càng khó dự đoán hành vi, khó gỡ lỗi, và tốn kém hơn (nhiều lượt gọi mô hình hơn cho cùng một nhiệm vụ).

2. Thành phần cốt lõi của một agent

2.1. Vòng lặp reasoning – acting (ReAct)

Phần lớn agent hiện nay vận hành theo một vòng lặp được mô tả và đặt tên trong nghiên cứu ReAct (Yao và cộng sự, 2023) đã được nhắc sơ lược ở tài liệu Prompt Engineering, giờ là trọng tâm của tài liệu này: mô hình suy luận (reasoning) về bước tiếp theo cần làm, thực hiện một hành động (action) nếu cần thường là gọi một công cụ bên ngoài quan sát (observation) kết quả trả về, rồi tiếp tục suy luận dựa trên quan sát đó. Vòng lặp này lặp lại cho đến khi mô hình xác định đã đủ thông tin để hoàn thành nhiệm vụ.



Hình 1. Vòng lặp reasoning – acting (ReAct): mô hình suy luận, hành động qua công cụ, quan sát kết quả, và lặp lại cho đến khi hoàn thành nhiệm vụ.

Điểm khác biệt so với một lượt hỏi-đáp thông thường là số vòng lặp không cố định trước agent có thể cần một bước, hoặc vài chục bước, tùy độ phức tạp của nhiệm vụ và những gì phát sinh trong quá trình thực hiện.

2.2. Function calling / tool use

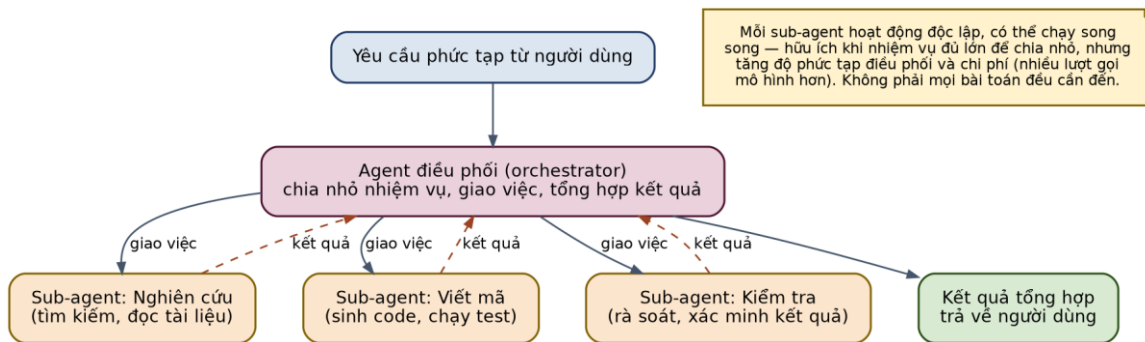
Để một agent hành động được, mô hình cần một cơ chế gọi công cụ bên ngoài có cấu trúc gọi là function calling hay tool use: đội phát triển mô tả cho mô hình biết những công cụ nào có sẵn (tên, mục đích, tham số cần thiết), mô hình khi cần sẽ sinh ra một yêu cầu gọi công cụ đúng định dạng (thường là JSON), hệ thống thực thi lệnh gọi đó thực sự (tìm kiếm web, truy vấn cơ sở dữ liệu, chạy đoạn mã, gọi API nội bộ...), rồi trả kết quả lại cho mô hình dưới dạng quan sát. Chất lượng của phần mô tả công cụ (tên rõ ràng, mô tả đủ chi tiết, tham số hợp lý) ảnh hưởng trực tiếp đến việc mô hình có chọn đúng công cụ và dùng đúng cách hay không đây thực chất là một dạng prompt engineering áp dụng cho công cụ, không chỉ cho câu hỏi. Một chuẩn mở đáng chú ý gần đây là Model Context Protocol (MCP), giúp chuẩn hoá cách agent kết nối với nhiều nguồn công cụ/dữ liệu khác nhau mà không cần tích hợp riêng lẻ cho từng nguồn.

2.3. Bộ nhớ (memory)

Một agent xử lý nhiệm vụ phức tạp cần "nhớ" những gì đã xảy ra trong quá trình thực hiện. Bộ nhớ ngắn hạn nằm trong cửa sổ ngữ cảnh của lượt gọi hiện tại toàn bộ lịch sử suy luận/hành động/quan sát từ đầu nhiệm vụ, giúp mô hình không lặp lại hành động đã thử hay quên mất mục tiêu ban đầu. Với nhiệm vụ dài hoặc agent cần duy trì trạng thái qua nhiều phiên làm việc khác nhau, cần thêm bộ nhớ dài hạn thường lưu ngoài, dưới dạng tóm tắt hoặc entry có cấu trúc, được truy xuất lại khi cần (một dạng áp dụng RAG cho chính lịch sử hoạt động của agent, ý tưởng này được minh hoạ rõ trong nghiên cứu về generative agents mô phỏng hành vi con người của Park và cộng sự, 2023).

3. Kiến trúc đa tác nhân (multi-agent)

Với nhiệm vụ đủ lớn hoặc cần nhiều loại chuyên môn khác nhau, một agent duy nhất xử lý mọi việc có thể trở nên công kênh ngừ cảnh phải chứa quá nhiều thông tin không liên quan đến bước hiện tại, làm mô hình dễ "lạc hướng". Kiến trúc đa tác nhân giải quyết vấn đề này bằng cách chia nhiệm vụ cho nhiều agent chuyên biệt, phối hợp bởi một agent điều phối (orchestrator):



Hình 2. Kiến trúc đa tác nhân: agent điều phối chia nhỏ nhiệm vụ, giao cho các sub-agent chuyên biệt, rồi tổng hợp kết quả.

Đánh đổi cốt lõi giống hệt nguyên tắc task decomposition đã nói ở tài liệu Prompt Engineering: chia nhỏ giúp mỗi agent con tập trung, dễ gỡ lỗi và có thể chạy song song, nhưng tăng độ trễ tổng thể (nhiều lượt gọi mô hình hơn) và độ phức tạp điều phối (agent điều phối cần tổng hợp đúng kết quả từ nhiều nguồn, xử lý trường hợp một sub-agent thất bại). Nguyên tắc chung tương tự phần 1: chỉ chuyển sang kiến trúc đa tác nhân khi một agent đơn đã cho thấy không đủ, không phải như một lựa chọn mặc định ngay từ đầu.

4. Đánh giá và gỡ lỗi agent

Đánh giá agent khó hơn đánh giá một lượt hỏi-đáp đơn (đã bàn ở tài liệu Đánh giá mô hình của loạt bài) vì có thêm một chiều mới: không chỉ câu trả lời cuối cùng có đúng không, mà cả quá trình đi đến câu trả lời đó có hợp lý không — một agent có thể vô tình đạt đúng kết quả qua một chuỗi hành động không tối ưu (gọi công cụ thừa, đi vòng), hoặc ngược lại, thất bại dù đã chọn đúng chiến lược vì một bước thực thi giữa chừng bị lỗi.

Vì vậy, đánh giá agent hiệu quả thường cần log lại toàn bộ vết (trace) suy luận hành động quan sát cho mỗi lượt chạy, không chỉ kết quả cuối, để có thể rà soát agent "nghĩ" gì ở từng bước và xác định chính xác bước nào gây lỗi khi có sự cố — cùng nguyên tắc cô lập lỗi theo thành phần đã nói ở tài liệu Đánh giá mô hình, áp dụng cho từng bước trong chuỗi hành động thay vì từng thành phần hệ thống tĩnh.

5. Rủi ro & giới hạn thực tế

5.1. Lỗi tích lũy qua nhiều bước

Nếu xác suất đúng ở mỗi bước riêng lẻ là 95% nghe có vẻ cao thì xác suất hoàn thành đúng cả một chuỗi 10 bước liên tiếp giảm xuống chỉ còn khoảng 60% (0.95^{10}), vì lỗi ở bất kỳ bước nào cũng có thể làm lệch hướng toàn bộ phần còn lại của nhiệm vụ. Đây là rủi ro toán học cơ bản của mọi hệ thống nhiều bước, và là lý do agent xử lý nhiệm vụ càng dài càng dễ thất bại theo cách khó lường, kể cả khi mô hình nền rất mạnh. Giảm thiểu bằng cách: giữ số bước cần thiết ở mức tối thiểu hợp lý, thêm các điểm kiểm tra (checkpoint) giữa chừng để phát hiện sớm khi agent đi sai hướng thay vì để nó tiếp tục hàng chục bước rồi mới phát hiện, và cho phép agent tự xác minh lại một phần kết quả trước khi coi nhiệm vụ đã hoàn thành.

5.2. Quyền hạn & an toàn

Khi một agent có thể tự gọi công cụ và hành động thực sự không chỉ sinh văn bản rủi ro vận hành tăng lên đáng kể so với một chatbot chỉ trả lời câu hỏi: một hành động sai (xóa nhầm dữ liệu, gửi nhầm email, thực thi mã độc hại) có hậu quả thực tế, không chỉ là một câu trả lời tệ. Nguyên tắc quan trọng nhất, đã được nhắc đến khi bàn về prompt injection ở tài liệu Prompt Engineering, càng cấp thiết hơn với agent: nguyên tắc ít đặc quyền nhất (least privilege) chỉ cấp cho agent đúng những quyền/công cụ cần thiết cho nhiệm vụ, không hơn. Bổ sung thêm: yêu cầu xác nhận của con người (human-in-the-loop) trước các hành động không thể hoàn tác hoặc có rủi ro cao (xóa dữ liệu, giao dịch tài chính, gửi thông tin ra ngoài); giới hạn số bước/thời gian tối đa cho một lượt chạy để tránh agent lặp vô hạn; và luôn log đầy đủ để có thể truy vết và hoàn tác khi cần.

6. Checklist nhanh cho đội ngũ

- Nhiệm vụ có thực sự cần một agent tự chủ, hay một workflow cố định đơn giản hơn đã đủ giải quyết?
- Mô tả công cụ (tên, mục đích, tham số) đã đủ rõ ràng để mô hình chọn đúng công cụ và dùng đúng cách chưa?
- Agent có được cấp đúng và chỉ đúng những quyền/công cụ cần thiết cho nhiệm vụ (least privilege) không?
- Các hành động không thể hoàn tác hoặc rủi ro cao có yêu cầu xác nhận của con người trước khi thực thi không?
- Có giới hạn số bước/thời gian tối đa cho một lượt chạy để tránh lặp vô hạn hoặc chi phí vượt kiểm soát?
- Có log lại toàn bộ vết suy luận hành động quan sát để gỡ lỗi khi agent thất bại, không chỉ log kết quả cuối?
- Trước khi chuyển sang kiến trúc đa tác nhân, đã xác nhận một agent đơn thực sự không đủ xử lý nhiệm vụ chưa?

Đọc thêm

Tài liệu này khép lại loạt bài với các nguyên tắc nền tảng của AI agent. Nếu bạn muốn đào sâu hơn với một góc nhìn hệ thống, đầy đủ ví dụ mã nguồn và case study thực chiến về xây dựng agent trong bối cảnh ứng dụng production trên nền foundation model, cuốn sách sau là một nguồn tham khảo chuyên sâu đáng đọc:

"AI Engineering: Building Applications with Foundation Models" Chip Huyen, O'Reilly Media, 2025.

Bản tiếng Việt phát hành bởi Times, có bán tại các nhà sách và sàn thương mại điện tử.

Tài liệu bạn đang đọc là nội dung gốc, biên soạn độc lập dựa trên kiến thức chung của ngành và các nguồn công khai liệt kê bên dưới không phải bản dịch hay tóm tắt của cuốn sách trên.

Tài liệu tham khảo

- Yao, S. et al. "ReAct: Synergizing Reasoning and Acting in Language Models." ICLR, 2023 (arXiv:2210.03629).
- Anthropic. "Building Effective Agents." anthropic.com/research, 2024.
- Park, J. et al. "Generative Agents: Interactive Simulacra of Human Behavior." UIST, 2023 (arXiv:2304.03442).
- Wang, L. et al. "A Survey on Large Language Model based Autonomous Agents." arXiv:2308.11432, 2024.
- Anthropic. "Model Context Protocol (MCP) specification." modelcontextprotocol.io, 2026.
- OpenAI. "Function calling guide." platform.openai.com/docs, 2026.